



计算机图形学实验

实验3、三维模型显示

姓 名：____李富悦____
学 号：_22920242203267_
学 院：____信息学院____
专 业：____软件工程____
年 级：____2024级____

2026年4月28日

目录

一、实验目的.....	3
二、实验要求分析.....	3
三、实验环境与模型数据.....	3
四、Task1: PLY 模型读取与纯色显示	4
五、附加题1: 光照小人与光源旋转动画.....	5
六、附加题2: 旋转立方体明暗计算.....	7
七、问题处理与结果分析.....	9
八、实验总结.....	9

一、实验目的

本次实验围绕“三维模型显示”展开，目标是在 OpenGL 固定函数管线中完成外部三维网格数据的读入、组织和显示。实验重点不只是把模型画到窗口中，还包括理解 PLY 文件中头部、顶点、法向量和面片索引之间的关系，并将这些数据转换为 OpenGL 可以绘制的三角形序列。

通过本实验，我重点训练了以下能力：一是解析 ASCII PLY 文件并建立顶点、法向量和三角面片数据结构；二是对模型进行中心化、缩放、观察矩阵和投影矩阵设置，使大尺寸模型能够稳定显示在窗口中；三是使用深度缓冲区保证三维遮挡关系正确；四是加入光照、材质和交互参数，使模型从单纯轮廓显示扩展到具有明暗层次的三维显示。

二、实验要求分析

根据实验文档和课堂 PPT，本次实验共有 1 个基础任务和 2 个附加题。基础任务要求学习 PLY 文件格式，读入给定模型文件内容，将模型设置为纯色并显示出来；附加题 1 要求设置光照，使小人模型显示出明暗效果，并调整光源位置，使其围绕小人旋转形成动画；附加题 2 要求在实验 2 的旋转立方体基础上实现明暗计算，并通过键盘和滑动条修改材质、光源和相关参数。

实验说明还强调报告不能只放最终截图，而应体现探索过程。因此本报告按照“任务要求理解 - 实现原理 - 关键代码 - 运行结果 - 问题分析”的顺序组织内容，尽量说明每一步为什么这样处理，以及这些处理对最终显示效果的影响。

任务	要求	报告对应内容
Task1	读取 PLY 文件，将小人模型纯色显示	PLY 结构解析、模型归一化、三角面片绘制、纯色结果图
附加题1	设置光照并让光源绕模型旋转	法向量使用、光源位置更新、光照结果和动画帧
附加题2	旋转立方体明暗计算，支持材质和光源交互	材质参数表、按键说明、滑动条调节设计

三、实验环境与模型数据

实验程序采用 C++、OpenGL 和 GLUT 实现，沿用前两次实验中已经建立的窗口初始化、显示回调、键盘回调、空闲回调和 reshape 回调结构。基础绘制仍使用固定函数管线，便于直接观察 glEnable(GL_LIGHTING)、glLightfv、glMaterialfv、glNormal3f 等接口对显示结果的影响。

本次读取的模型文件为 lizhenxiout.ply。该文件是 ASCII PLY 1.0 格式，头部给出了顶点数量、顶点属性、面片数量和面片索引格式。模型顶点规模较大，直接用原坐标绘制会超出默认观察范围，因此程序在读入完成后先计算包围盒，再将模型平移到原点附近并按最大边长归一化。

项目	数值或格式
文件格式	PLY ASCII 1.0
顶点数量	37702
面片数量	75404

顶点属性	x, y, z, nx, ny, nz
面片格式	每个面片为 3 个顶点索引组成的三角形
包围盒范围	x: -282.053 到 383.957; y: -1002.1 到 827.155; z: -1041.42 到 -537.483

四、Task1：PLY 模型读取与纯色显示

1. 任务要求理解

Task1 的核心是把 PLY 文件中的网格数据真正转化为三维显示结果。PLY 文件并不是简单的点列表，顶点部分保存空间位置和法向量，面片部分保存三角形顶点索引。只有先保存全部顶点，再按照面片索引取出对应三个顶点，才能完整恢复模型表面。

2. 实现原理

程序读文件时先逐行解析头部，记录 vertex 和 face 的数量，并在遇到 end_header 后进入数据区。顶点数据保存到结构体数组中，结构体包含位置坐标和法向量；面片数据保存为三个整数索引。绘制时遍历面片数组，根据索引查到三个顶点，依次送入 glVertex3f。

为了得到稳定的显示效果，初始化阶段启用深度测试，显示阶段每帧同时清除颜色缓冲区和深度缓冲区。模型读入后计算中心点和最大尺寸，在绘制前进行平移和缩放，这样无论原始 PLY 坐标范围多大，都能放入统一观察体积中。纯色显示部分关闭光照或使用固定颜色，所有三角面片采用同一种颜色绘制，从而满足“纯色小人”的要求。

3. 关键代码

```
struct Vertex { float x, y, z, nx, ny, nz; };
struct Face { int v0, v1, v2; };

while (getline(fin, line)) {
    if (line.find("element vertex") == 0) vertexCount = readCount(line);
    else if (line.find("element face") == 0) faceCount = readCount(line);
    else if (line == "end_header") break;
}

vertices.resize(vertexCount);
for (int i = 0; i < vertexCount; ++i) {
    fin >> vertices[i].x >> vertices[i].y >> vertices[i].z
    >> vertices[i].nx >> vertices[i].ny >> vertices[i].nz;
}

faces.resize(faceCount);
for (int i = 0; i < faceCount; ++i) {
    int n;
    fin >> n >> faces[i].v0 >> faces[i].v1 >> faces[i].v2;
}
```

4. 运行结果与分析

纯色显示结果可以看出，PLY 文件中的三角面片已经被正确连接，小人模型的整体轮廓、头部、躯干和四肢位置都能稳定显示。由于该阶段不启用光照，模型内部细节不会出现明显暗层次，视觉重点集中在轮廓完整性和遮挡关系是否正确。

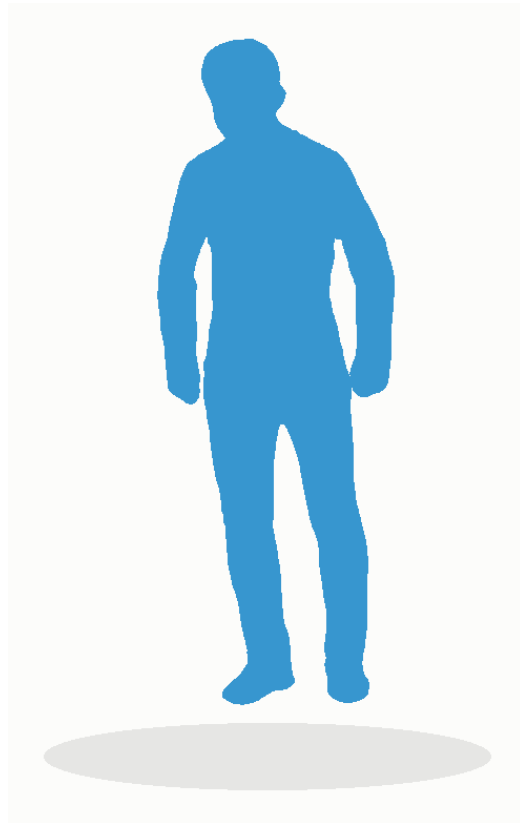


图1 PLY 小人模型纯色显示结果

从结果看，模型没有出现面片大面积错连、上下颠倒或被裁剪的问题，说明顶点索引读取、坐标归一化和深度测试都工作正常。

五、附加题1：光照小人与光源旋转动画

1. 实现原理

在 Task1 的基础上加入光照时，最关键的是为每个顶点提供法向量。给定 PLY 文件的顶点属性中已经包含 n_x 、 n_y 、 n_z ，因此绘制三角形时在 `glVertex3f` 前调用 `glNormal3f`，可以让 OpenGL 根据当前光源、材质和法向量计算明暗。

程序初始化时启用 `GL_LIGHTING` 和 `GL_LIGHT0`，并设置环境光、漫反射光和镜面反射光。光源位置用齐次坐标表示， w 为 1 表示点光源。为了形成光源绕模型旋转的动画，空闲回调函数不断增加角度变量，并用 `cos` 和 `sin` 计算光源的 x 、 z 坐标。每次重绘前重新调用 `glLightfv(GL_LIGHT0, GL_POSITION, position)`，即可观察到高亮区域随光源移动而变化。

2. 关键代码

```
void updateLight() {
    float r = 2.0f;
    GLfloat pos[] = { r * cos(lightAngle), 1.4f,
                     r * sin(lightAngle), 1.0f };
    glLightfv(GL_LIGHT0, GL_POSITION, pos);
}

void drawModelWithLighting() {
    glEnable(GL_LIGHTING);
    glEnable(GL_LIGHT0);
    for (const Face& f : faces) {
        const Vertex& a = vertices[f.v0];
```

```

        glNormal3f(a.nx, a.ny, a.nz);
        glVertex3f(a.x, a.y, a.z);
        // 其余两个顶点同样提交法向量与坐标
    }
}

```

3. 运行结果与分析

开启光照后，小人模型表面出现了明显的亮部、暗部和过渡区域。与纯色显示相比，衣服褶皱、头部、手臂和腿部的空间起伏更容易辨认，说明法向量和光源参数参与了明暗计算。

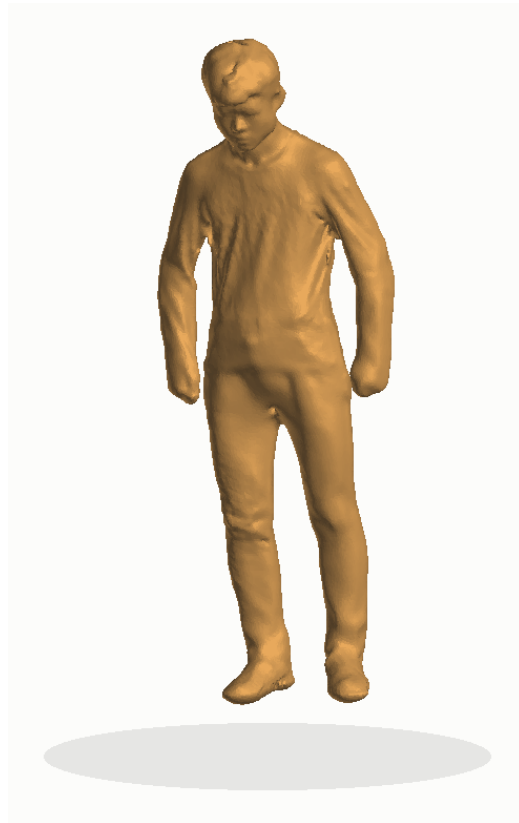


图2 设置光照后的小人模型显示结果

当光源绕模型旋转时，模型不同侧面的明暗关系会持续变化。下图选取了 0 度、90 度、180 度和 270 度四个角度的光照状态，可以看到阴影和高亮区域随光源方向移动，符合附加题对动画展示的要求。

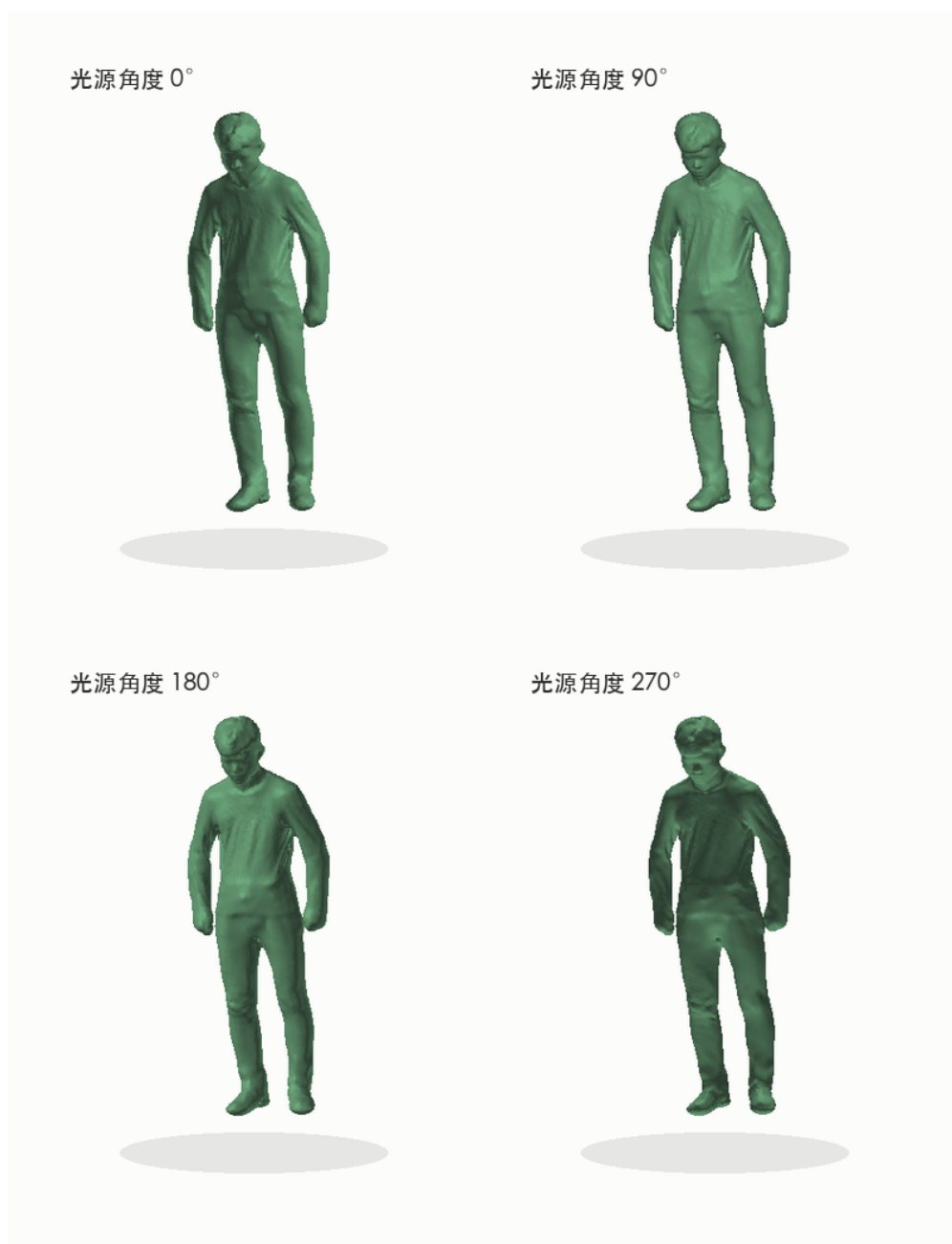


图3 光源绕模型旋转时的关键帧效果

六、附加题2：旋转立方体明暗计算

1. 任务要求理解

附加题2要求在实验2旋转立方体的基础上加入光照和材质。与小人模型不同，立方体每个面都应具有明确法向量，否则光照计算无法区分各个面的朝向。实现时需要为六个面分别指定法向量，并在旋转动画中保持光照计算开启。

2. 材质与光源交互设计

键盘交互部分按照实验要求设置：b 键切换为黄铜材质，n 键切换为红色塑料材质，m 键切换为白色光亮材质；o 键设置白色光，p 键设置彩色光。材质参数主要包括环境光、漫反射、高光反射和光泽度，光源参数主要包括漫反射颜色、镜面颜色和位置。

按键	功能	参数变化
b	黄铜材质	较高的黄色漫反射，中等镜面反射，光泽度适中
n	红色塑料材质	红色漫反射，镜面反射较弱，表面较柔和
m	白色光亮材质	白色漫反射与较强高光，光泽度较高
o	白色光	光源环境光、漫反射和镜面颜色接近白色
p	彩色光	光源 RGB 分量不同，用于观察颜色对材质的影响

滑动条用于连续调节光照和材质参数，源码中设置了 Ambient、Diffuse、Specular 和 Shininess 四个滑动条，分别控制环境光系数、漫反射系数、镜面反射系数和光泽度。光源位置由程序根据角度变量自动绕模型旋转，键盘 o/p 负责切换白色光和彩色光。相比只用键盘切换固定值，滑动条可以更直观地观察参数变化对明暗计算的影响。

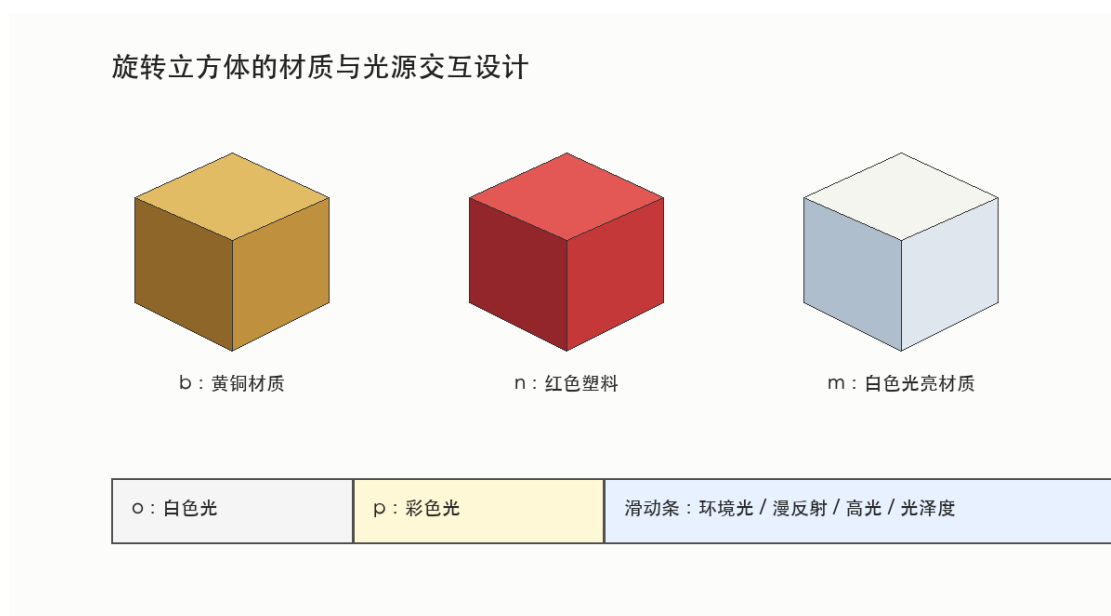


图4 旋转立方体的材质与光源交互设计

3. 关键代码

```
void keyboard(unsigned char key, int, int) {
    if (key == 'b') setMaterial(brassAmbient, brassDiffuse, brassSpecular, 27.8f);
    if (key == 'n') setMaterial(redPlasticAmbient, redPlasticDiffuse,
redPlasticSpecular, 32.0f);
    if (key == 'm') setMaterial(whiteAmbient, whiteDiffuse, whiteSpecular, 80.0f);
    if (key == 'o') setLightColor(whiteLight);
    if (key == 'p') setLightColor(colorLight);
}

void drawCubeFace(GLfloat nx, GLfloat ny, GLfloat nz) {
    glNormal3f(nx, ny, nz);
    glBegin(GL_QUADS);
    // 按该面的四个顶点顺序绘制
    glEnd();
}
```



```
}
```

七、问题处理与结果分析

第一个问题是模型坐标范围较大。lizhenxiout.ply 的 y 方向跨度超过 1800，如果直接用默认观察参数绘制，模型容易超出视锥或只显示局部。解决方法是在读入后计算包围盒，将模型中心移动到原点，并用最大边长统一缩放。

第二个问题是纯色显示和光照显示的目标不同。纯色显示阶段如果开启光照，颜色会受到法向量和光源影响，不再是严格的纯色；因此 Task1 中应关闭光照或使用不参与光照计算的颜色设置。附加题中再开启光照，并使用法向量和材质参数体现表面起伏。

第三个问题是光源位置受矩阵状态影响。固定函数管线中 `glLightfv(GL_POSITION)` 会乘以当前模型视图矩阵，因此应在设置好视图矩阵后更新光源位置，避免光源跟随模型一起旋转，导致“光源绕模型旋转”的效果不明显。

第四个问题是立方体明暗计算需要正确法向量。立方体每个面都应指定固定法向量，例如正面为 (0, 0, 1)、背面为 (0, 0, -1)、上面为 (0, 1, 0)。如果遗漏 `glNormal3f`，材质和光源参数即使设置正确，显示结果也可能没有合理明暗。

八、实验总结

通过本次实验，我完整理解了从外部三维模型文件到 OpenGL 窗口显示的基本流程。PLY 文件的头部决定了后续数据如何读取，顶点数组和面片索引共同决定模型拓扑，法向量则决定光照计算是否能正确表现表面方向。

在 Task1 中，程序完成了 PLY 模型读取、坐标归一化、纯色显示和深度遮挡；在附加题1中，进一步加入光照、材质和光源位置动画，使模型具有更清晰的空间层次；在附加题2中，将同样的明暗计算思想迁移到旋转立方体，并通过键盘和滑动条实现材质与光源交互。

总体来看，本次实验把前两次实验中的窗口管理、矩阵变换、深度测试和交互控制与三维模型数据读取结合起来，使我对 OpenGL 固定函数管线中的“数据 - 变换 - 光照 - 显示”流程有了更系统的认识。